

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#01)

Apply modern simulation tools ModelSim and Logisim to implement basic processor components

Student Name: Fatima Tariq Roll No: 24i-6557 Section: CE-A

Date performed: 24. August 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1: Write Verilog codes for Basic gates using structural (Gate level) modeling and verify results.

Description:

In this task waveform of different gates were plotted in ModelSim using Gate level Verilog programming. Where the first part of each code represents the design part where we tell the computer which is our input and which is our output, while testbench is the part in each code which is simulated to check the waveform of each gate to see if it verifies the gates properties and logic table, here reg is used for input and wire is used for output.

Code:

And Gate:

```
module logicGates(a,b,c);
input a;
input b;
output c;
and a1(c,a,b);
endmodule
module testbench();
reg x;
reg y;
wire z;
logicGates uut(x,y,z);
initial
begin
x=0; y=0;
#50 x=0; y=1;
#50 x=1; y=0;
#50 x=1; y=1;
end
endmodule
```

OR Gate:

```
module task01_part2(a,b,c);
input a;
input b;
output c;
or a1(c,a,b);
endmodule
module testbench();
reg x;
reg y;
wire z;
task01_part2 LS(x,y,z);
initial
begin
```

```
x=0;y=0;
#50 x=0;y=1;
#50 x=1;y=0;
#50 x=1;y=1;
end
endmodule
```

NOT Gate:

```
module part03(a,c);
input a;
output c;
not a1(c,a);
endmodule
module testbench();
reg x;
wire z;
part03 LS(x,z);
initial
begin
x=0;
#50 x=1;
end
endmodule
```

NAND Gate:

```
module part04(a,b,c);
input a;
input b;
output c;
nand a1(c,a,b);
endmodule
module testbench_NAND();
reg x;
reg y;
wire z;
part04 LS(x,y,z);
initial
begin
x=0; y=0;
#50 x=0; y=1;
#50 x=1; y=0;
#50 x=1; y=1;
end
endmodule
```

NOR Gate:

```
module part05(a,b,c);
input a;
input b;
output c;
nor a1(c,a,b);
endmodule
module testbench_NOR();
```

```
reg x;  
reg y;  
wire z;  
part05 LS(x,y,z);  
initial  
begin  
    x=0; y=0;  
#50 x=0; y=1;  
#50 x=1; y=0;  
#50 x=1; y=1;  
end  
endmodule
```

XOR Gate:

```
module part06(a,b,c);  
input a;  
input b;  
output c;  
xor a1(c,a,b);  
endmodule  
module testbench_XOR();  
reg x;  
reg y;  
wire z;  
part06 LS(x,y,z);  
initial  
begin  
    x=0; y=0;  
#50 x=0; y=1;  
#50 x=1; y=0;  
#50 x=1; y=1;  
end  
endmodule
```

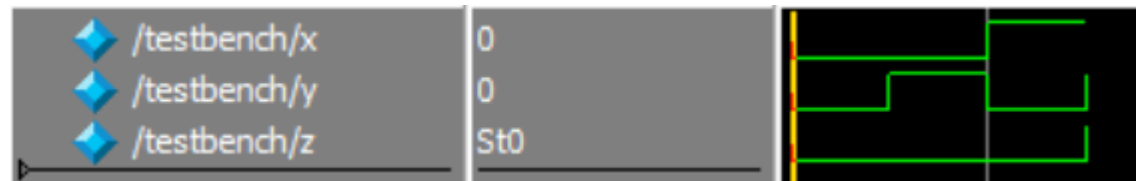
XNOR Gate:

```
module part07(a,b,c);  
input a;  
input b;  
output c;  
xnor a1(c,a,b);  
endmodule  
module testbench_XNOR();  
reg x;  
reg y;  
wire z;  
part07 LS(x,y,z);  
initial  
begin  
    x=0; y=0;  
#50 x=0; y=1;  
#50 x=1; y=0;  
#50 x=1; y=1;  
end
```

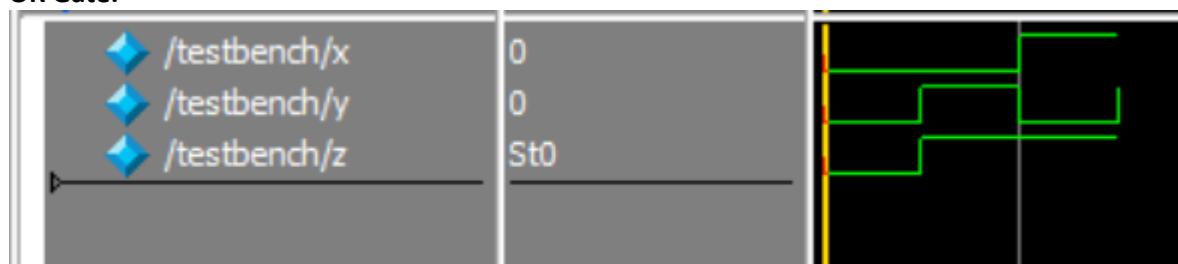
endmodule

Output:

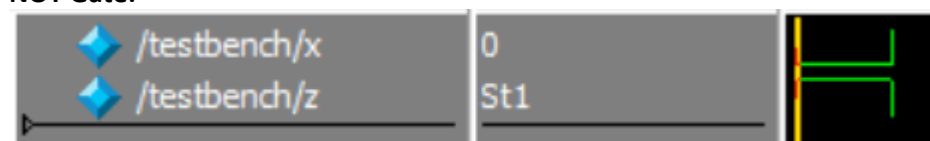
And Gate:



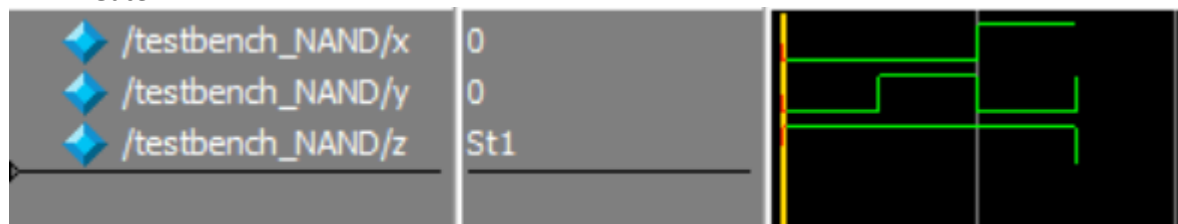
OR Gate:



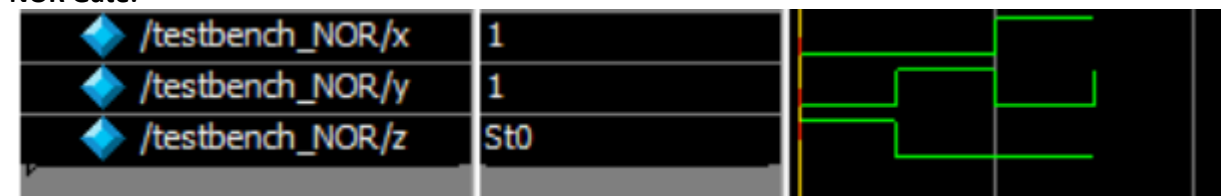
NOT Gate:



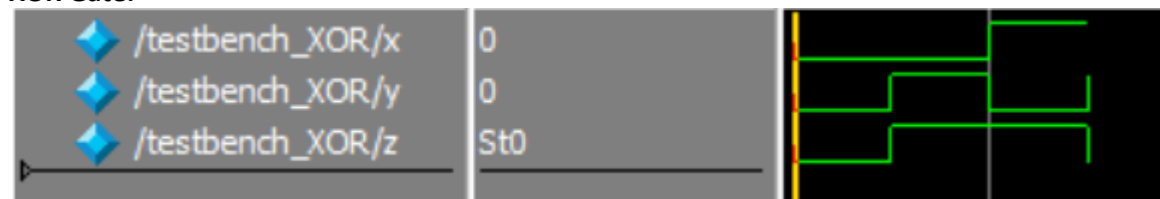
NAND Gate:



NOR Gate:



XOR Gate:



XNOR Gate:



Task 2: Write Verilog codes for Basic gates in Dataflow modeling and verify results using testbench.

Description:

In this task waveform of different gates were plotted in ModelSim using Data flow level Verilog programming. Where the first part of each code represents the design part where we tell the computer which is our input and which is our output, while testbench is the part in each code which is simulated to check the waveform of each gate to see if it verifies the gates properties and logic table, here reg is used for input and wire is used for output.

Code:

AND Gate:

```
module task02part01(a,b,c);
input a;
input b;
output c;
assign c = a & b;
endmodule

module testbench01();
reg x;
reg y;
wire z;
task02part01 LS(x,y,z);
initial
begin
    x=0; y=0;
    #50 x=0; y=1;
    #50 x=1; y=0;
    #50 x=1; y=1;
end
endmodule
```

OR Gate:

```
module task02part02(a,b,c);
input a;
input b;
output c;
assign c = a | b;
endmodule

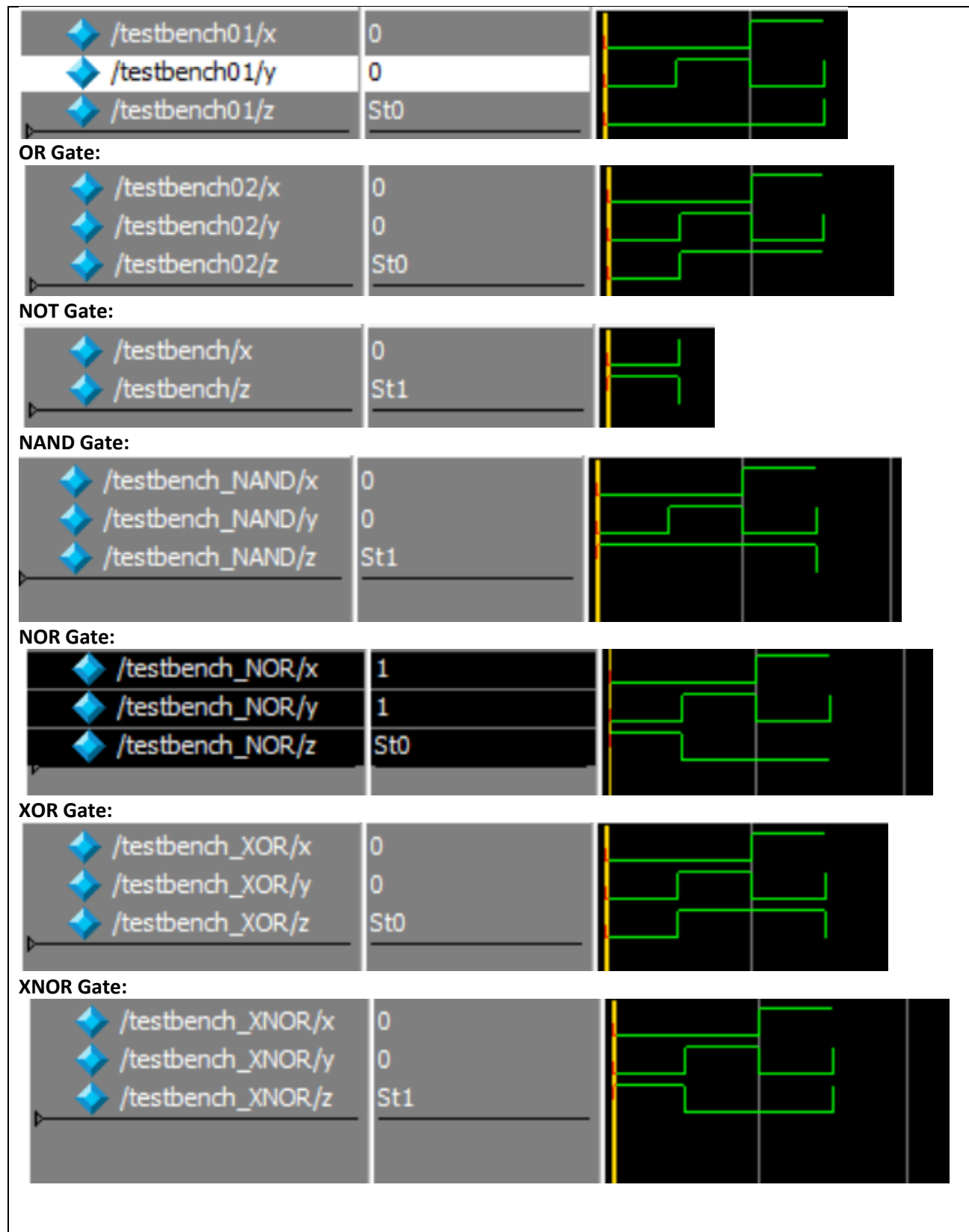
module testbench02();
reg x;
reg y;
wire z;
task02part02 LS(x,y,z);
initial
begin
    x=0; y=0;
    #50 x=0; y=1;
    #50 x=1; y=0;
    #50 x=1; y=1;
end
endmodule
```

```
end
endmodule
NOT Gate:
module task02part03(a,c);
input a;
output c;
assign c = ~a;
endmodule
module testbench();
reg x;
wire z;
task02part03 LS(x,z);
initial
begin
    x=0;
    #50 x=1;
end
endmodule
NAND Gate:
module task02part04(a,b,c);
input a;
input b;
output c;
assign c = ~(a & b);
endmodule
module testbench_NAND();
reg x;
reg y;
wire z;
task02part04 LS(x,y,z);
initial
begin
    x=0; y=0;
    #50 x=0; y=1;
    #50 x=1; y=0;
    #50 x=1; y=1;
end
endmodule
NOR Gate:
module task02part05(a,b,c);
input a;
input b;
output c;
assign c = ~(a | b);
endmodule
module testbench_NOR();
reg x;
reg y;
wire z;
task02part05 LS(x,y,z);
initial
begin
    x=0; y=0;
```

```
#50 x=0; y=1;
#50 x=1; y=0;
#50 x=1; y=1;
end
endmodule
XOR Gate:
module task02part06(a,b,c);
input a;
input b;
output c;
assign c = a ^ b;
endmodule
module testbench_XOR();
reg x;
reg y;
wire z;
task02part06 LS(x,y,z);
initial
begin
    x=0; y=0;
    #50 x=0; y=1;
    #50 x=1; y=0;
    #50 x=1; y=1;
end
endmodule
XNOR Gate:
module task02part07(a,b,c);
input a;
input b;
output c;
assign c = ~(a ^ b);
endmodule
module testbench_XNOR();
reg x;
reg y;
wire z;
task02part07 LS(x,y,z);
initial
begin
    x=0; y=0;
    #50 x=0; y=1;
    #50 x=1; y=0;
    #50 x=1; y=1;
end
endmodule
```

Output:

AND Gate:



Task 3: Test the working of basic logic gates using logisim.

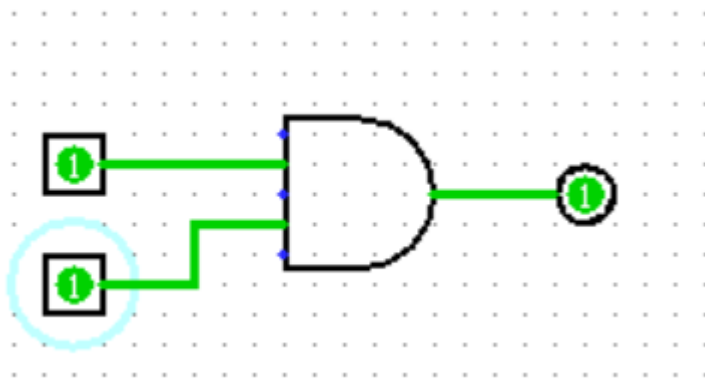
Description:

In this task Logisim was used to represent basic gates pictorially

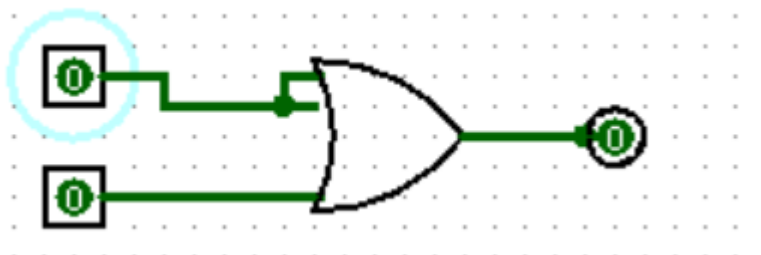
Code:

Output:

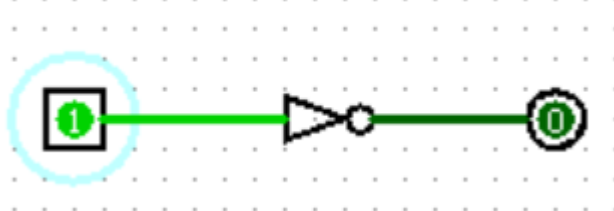
AND Gate:



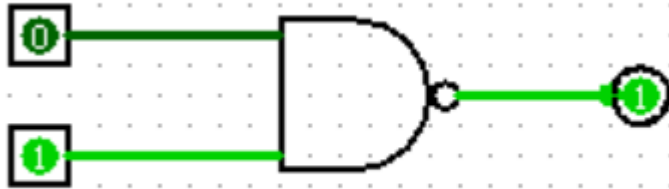
OR Gate:



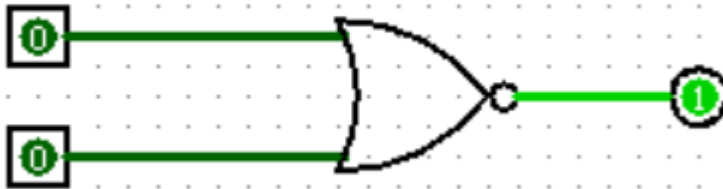
NOT Gate:



NAND Gate:



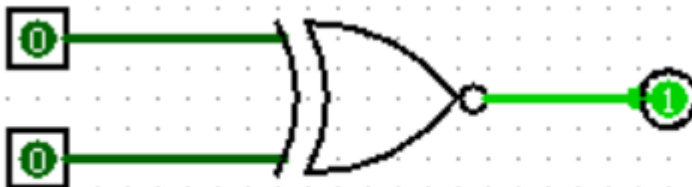
NOR Gate:



XOR Gate:



XNOR Gate:



Task 4: Write Verilog code for following circuit using gate level and data flow modeling and verify

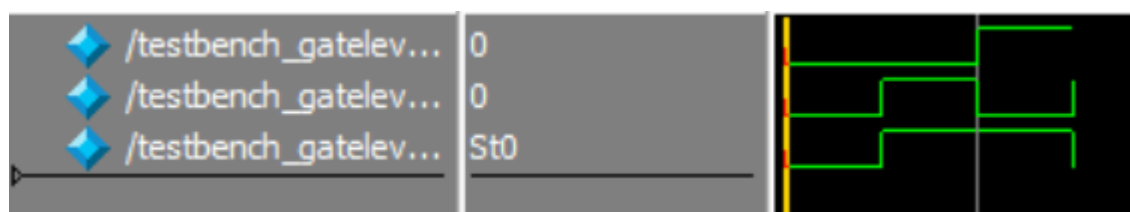
results using testbench.

Description:
In this task XOR gate was implemented using basic gates and its logic table was verified in Modelsim using both gate level and data flow level programming.
Code:
Gate Level: <pre>module task05(A, B, Z); input A; input B; output Z; wire notA; wire notB; wire and1; wire and2; not n1(notA, A); not n2(notB, B); and a1(and1, notA, B); and a2(and2, A, notB); or o1(Z, and1, and2); endmodule module testbench_gatelevel(); reg x; reg y; wire z; task05 LS(x, y, z); initial begin x = 0; y = 0; #50 x = 0; y = 1; #50 x = 1; y = 0; #50 x = 1; y = 1; end endmodule</pre> Data Flow Level: <pre>module task05(A, B, Z); input A; input B; output Z; assign Z = (~A & B) (A & ~B); endmodule module testbench_dataflow(); reg x; reg y; wire z; task05 LS(x, y, z); initial</pre>

```
begin
  x = 0; y = 0;
#50 x = 0; y = 1;
#50 x = 1; y = 0;
#50 x = 1; y = 1;
end
endmodule
```

Output:

Gate Level:



Data Flow Level:



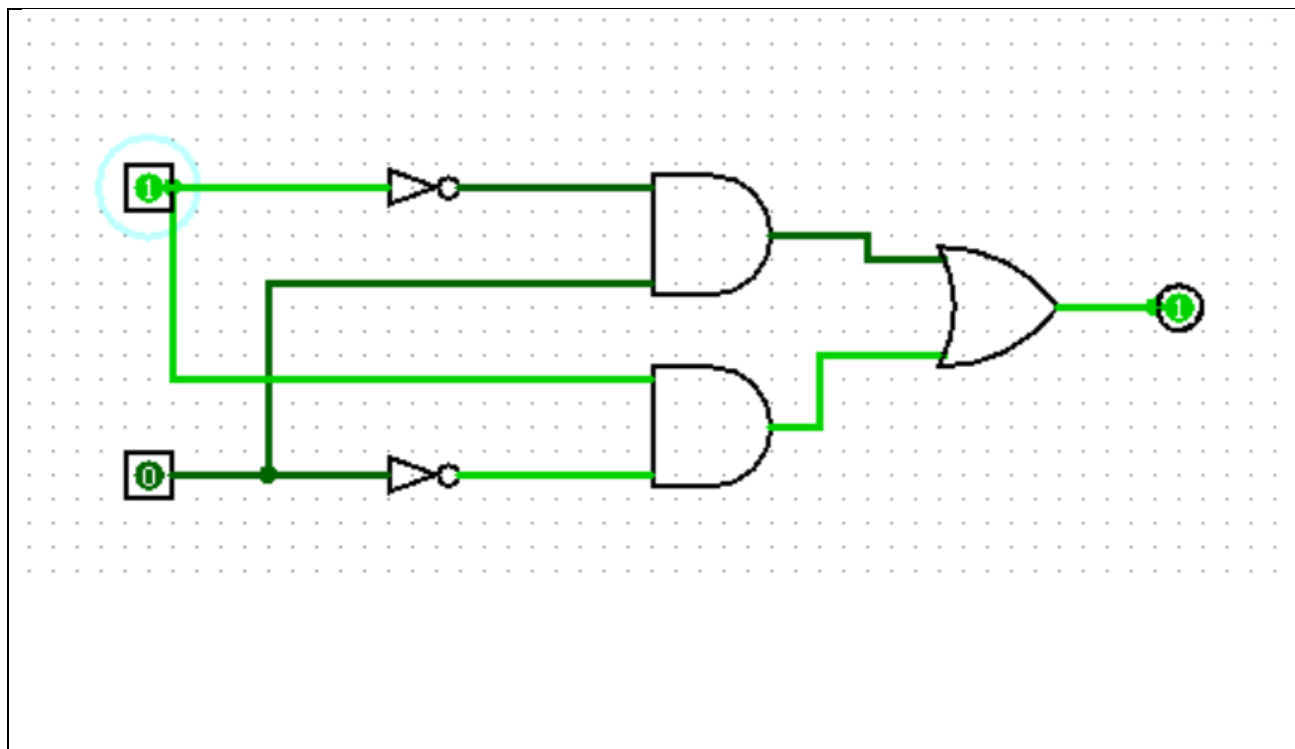
Task 5: Test the working of above circuit using logisim.

Description:

In this task the above XOR equation was verified in Logisim. different outputs were tested to verify the functionality of XOR Gate.

Code:

Output:



Task 6: Write Verilog code to find 1's complement of four bits using gate level and data flow modeling. Verify results using testbench.

Description:

In this Modelsim was used to implement a 1's complement of four bits input using both gate level and data flow level programming and results were verified for both using their testbenches.

Code:

Gate Level:

```
module task03part01(a,b,c,d,w,x,y,z);
input a,b,c,d;
output w,x,y,z;
not n1(w,a);
not n2(x,b);
not n3(y,c);
not n4(z,d);
endmodule
module testbench01();
reg a,b,c,d;
wire w,x,y,z;
task03part01 LS(a,b,c,d,w,x,y,z);
initial
begin
```

```
a=0; b=0; c=0; d=0;

#50 a=0; b=0; c=0; d=1;

#50 a=0; b=0; c=1; d=0;

#50 a=0; b=1; c=0; d=1;

#50 a=1; b=0; c=1; d=0;

#50 a=1; b=1; c=1; d=1;
end
endmodule
```

Data Flow Level:

```
module task03part01(a,b,c,d,w,x,y,z);
input a,b,c,d;
output w,x,y,z;
assign w = ~a;
assign x = ~b;
assign y = ~c;
assign z = ~d;
endmodule

module testbench02();
reg a,b,c,d;
wire w,x,y,z;
task03part01 LS(a,b,c,d,w,x,y,z);
initial
begin

    a=0; b=0; c=0; d=0;
    #50 a=0; b=0; c=0; d=1;
    #50 a=0; b=0; c=1; d=0;
    #50 a=0; b=1; c=0; d=1;
    #50 a=1; b=0; c=1; d=0;
    #50 a=1; b=1; c=1; d=1;
end
endmodule
```

Output:

GATE Level:

